SageFs: the REPL-first inner loop

SageFs is a warm F# Interactive daemon with a live project session, an MCP server, and a dashboard. It is the fast loop for this repo: an eval is milliseconds, a `dotnet build` is tens of seconds, and the full suites are slower still. The rule is simple:

> Prove a change in the SageFs REPL, then write it into the file. Run the full build and both unfiltered suites once, at the end.

For a cold start that also covers the .NET MCP and the `just` recipes, read `docs/agent-setup.typ`. This document goes deeper on SageFs itself.

SageFs does not replace the final gate. It removes the repeated builds that used to be the inner loop.

Installed pieces

Piece	Location
Binary	`/home/shalok/.dotnet/tools/sagefs` (version `0.6.923.0`)
OpenCode skill	`~/.config/opencode/skills/sagefs/SKILL.md` and sibling guides
OpenCode MCP (V2)	`~/.config/opencode/opencode.json`, `mcp.servers.sagefs`
OpenCode MCP (V1)	`~/.config/opencode/opencode-v1.json`, `mcp.sagefs`
Daemon state	`~/.SageFs/`
Logs	`~/.local/share/SageFs/mcp-serverYYYYMMDD.log`

The installed skill is the upstream `skills/sagefs/` folder. OpenCode advertises it as `sagefs`; loading it pulls in the REPL loop, session rules, design standards, testing rules, and troubleshooting.

PATH

`dotnet` is on the normal PATH, so SageFs finds it without extra setup. A shell started before the PATH change may still carry the old environment; open a new shell. The OpenCode MCP entries set `DOTNET_ROOT` and the same toolchain directories, so the MCP server does not depend on the shell that launched OpenCode.

First checks

```
sagefs check          # SDK, F# Interactive, project discovery, ports, daemon state
sagefs status         # daemon process identity, port, directory, session count
```

`check` is the cheap pre-flight. It reports the machine tier (this machine is `Constrained`, so waits are scaled) and finds the one project in the repo: `bases/scale-cli/scale.fsproj`.

The repository's `just mcp-check` is the fast gate: it checks that every configured server is enabled, resolves each server command, and confirms the daemon. It does not start the servers. `just mcp-verify` runs the full handshake once and caches its success until the config or a server binary changes; pass `force` to bypass the cache.

Start and inspect the daemon

```
sagefs --ttl 2h       # run in its own terminal
sagefs status
```

`--ttl` is a **idle** lifetime, not a guaranteed uptime. With no live sessions and no connected clients for that window, the daemon logs `self-terminating` and exits. A missing daemon is therefore normal,

not a fault. For the rules an agent must follow when starting, sharing, or stopping the daemon, see `AGENTS.md` section 0.

`just sage-ensure` is the workflow helper. It starts a daemon only when none is running, reuses this checkout's active session when one exists, and creates a project session when none is active. It reads the session state from the MCP `get_session_status` response and never stops or restarts a shared daemon.

Useful endpoints:

Endpoint	Purpose
`http://localhost:37749/`	MCP Streamable HTTP
`http://localhost:37749/sse`	MCP server-sent events
`http://localhost:37750/dashboard`	Live sessions, evals, events
`POST /api/sessions/stop`	Stop a session when the MCP tool needs confirmation

The loop

During alpha, this REPL loop is the whole verification path: do not run MSBuild, Fantomas, or a NativeAOT build. `just check` is the release gate, not a development step.

The loop has moved to the repository-root `AGENTS.md`, section 4 ("The SageFs inner loop"). That section carries the orient, red, green, persist, reload, re-verify, gate, and clean-up sequence, the time-saving rules, and the tool names.

Stale builds and the session policy

A session's worker loads assemblies. A source or `.fsproj` change does not hot-replace library code; the worker keeps serving the last successful build until a rebuild succeeds. `get_session_status` reports this as `Stale` (or `Rebuilding`). The stale build is not a SageFs defect: it is the .NET assembly lifecycle plus a workflow that evaluates before rebuilding.

1. **One long-lived session per worktree.** Prefer a solution session that loads the product and the test deployable together. Do not stop and recreate sessions for each project; each create loads the last build and starts the stale cycle again.
2. **Gate every eval and test on freshness.** Read `sourceState` first. If it is not `InSync`, call `hard_reset_fsi_session` with `rebuild=true` and wait for `BecameReady`. A green result from a stale session is not evidence.
3. **Batch edits, then rebuild once.** Make all source and `.fsproj` changes for a slice before the rebuild. A `.fsproj` change always forces a full graph rebuild and a worker respawn.
4. **A failed rebuild is a stop sign.** Fix the compile error and rebuild. Never treat the old build's tests as evidence while the new build is broken.
5. **Check the binary.** Run `just freshness-check` before executing a built product binary; a fresh test build does not imply a fresh base.
6. **Use the snippet path for type experiments.** `check_fsharp_code` and `send_fsharp_code` prove small expressions without a project rebuild; persist only after the snippet passes.

Proved in this repo

The integration was exercised against the real project, not a toy:

```
1 + 1;;
// val it: int = 2
```

```
ScaleFile.parse "specification: Every player that is alive can jump.";;
// val it: Result<ScaleDocument,string> =
//   Ok { Specification = "Every player that is alive can jump."
//        Refinements = [] }
```

The second eval proves the session loaded `bases/scale-cli/scale.fsproj` and its modules, not just a bare REPL. The session was stopped afterward; `sagefs status` reported `0 active`.

OpenCode wiring

V2 config (`~/.config/opencode/opencode.json`):

```
{
  "mcp": {
    "servers": {
      "sagefs": {
        "type": "local",
        "command": ["/home/shalok/.dotnet/tools/sagefs", "mcp"],
        "environment": {
          "DOTNET_ROOT": "/usr/share/dotnet",
          "PATH": "/usr/share/dotnet:/home/shalok/.dotnet/tools:{env:PATH}"
        }
      }
    }
  }
}
```

V1 config (`~/.config/opencode/opencode-v1.json`) uses `mcp.sagefs` directly instead of `mcp.servers`:

```
{
  "mcp": {
    "sagefs": {
      "type": "local",
      "command": ["/home/shalok/.dotnet/tools/sagefs", "mcp"],
      "enabled": true,
      "environment": {
        "DOTNET_ROOT": "/usr/share/dotnet",
        "PATH": "/usr/share/dotnet:/home/shalok/.dotnet/tools:{env:PATH}"
      }
    }
  }
}
```

Check the connection with:

```
opencode mcp list
```

OpenCode exposes the tools as `sagefs_*` (or through Code Mode as `tools.sagefs.*`). The skill tells the agent when to use them; the MCP instructions repeat the first-minute checklist and the loop.

Troubleshooting

Troubleshooting has moved to the repository-root `AGENTS.md`, section 12. This page keeps the SageFs reference material: installation, endpoints, the daemon model, and the OpenCode wiring.